

CodePod Documentation

Table of Contents (`_sidebar.md`)

layout: default

title: CodePod Documentation

- * [Introduction](introduction.md)
- * [What Is CodePod?] (what-is-codepod.md)
 - * [Vision & Use Cases] (what-is-codepod.md#vision--use-cases)
 - * [Hardware & Software Summary] (what-is-codepod.md#hardware--software-summary)
- * [Hardware Components] (hardware-components.md)
 - * [Bill of Materials] (hardware-components.md#bill-of-materials)
 - * [Assembly Guide] (hardware-components.md#assembly-guide)
- * [Software Stack] (software-stack.md)
 - * [OS Recommendation] (software-stack.md#os-recommendation)
 - * [Preinstalled Tools] (software-stack.md#preinstalled-tools)
 - * [Optional Packages] (software-stack.md#optional-packages)
- * [Getting Started in 5 Minutes] (getting-started.md)
- * [Deep Dive: Step-by-Step Setup] (deep-dive-setup.md)
 - * [Partitioning & Filesystem] (deep-dive-setup.md#partitioning--filesystem-tuning)
 - * [Security Hardening] (deep-dive-setup.md#firewall-and-security-hardening)
 - * [Reverse Proxy & HTTPS] (deep-dive-setup.md#reverse-proxy--https)
 - * [First Application] (deep-dive-setup.md#deploying-your-first-app)
- * [Configuration & Customization](configuration.md)
 - * [Environment Variables](configuration.md#environment-variables)
 - * [Adding Services](configuration.md#adding-new-services)
 - * [Domain & DNS Setup](configuration.md#domain--dns-setup)
- * [Maintenance & Troubleshooting](maintenance.md)
 - * [Updates & Backups](maintenance.md#automatic-updates)
 - * [Common Issues](maintenance.md#common-errors--fixes)

- * [Advanced Topics](advanced-topics.md)
 - * [Clustering](advanced-topics.md#clustering-multiple-codepods)
 - * [Custom Hardware](advanced-topics.md#building-custom-pcbhat)
 - * [API Integration](advanced-topics.md#integrating-with-external-apis)
 - * [FAQ](faq.md)
 - * [Contributing & Roadmap](contributing.md)
 - * [Appendix](appendix.md)
 - * [Command Cheat Sheet](appendix.md#useful-commands-cheat-sheet)
 - * [References](appendix.md#references--further-reading)
-

introduction.md

layout: default

title: Introduction

sidebar: _sidebar.md

Welcome to CodePod

CodePod transforms your Raspberry Pi Zero 2 W into a powerful, self-hosted development environment and home server. Whether you're a hobbyist exploring Linux, a student learning web development, or a small business owner needing affordable hosting, CodePod provides everything you need in a compact, energy-efficient package.

Why CodePod?

In an era of cloud dependency and subscription fatigue, CodePod brings computing back to your control. For less than \$50, you can build a complete development server that runs 24/7, consumes minimal power, and gives you hands-on experience with real-world infrastructure.

What You'll Learn

By following this documentation, you'll gain practical experience with:

- Linux system administration
- Docker containerization
- Reverse proxy configuration
- SSL/TLS certificate management
- Network security fundamentals
- Infrastructure as Code principles

Prerequisites

Before diving in, ensure you have:

- Basic familiarity with command line interfaces
- Comfort with text editors (nano, vim, or VS Code)
- Understanding of basic networking concepts
- Patience for troubleshooting (it's part of the learning!)

💡 ****New to Linux?**** Don't worry! This documentation assumes minimal prior knowledge and explains concepts as we go.

Getting Help

- ****GitHub Issues****: Report bugs or request features
- ****Community Forum****: Ask questions and share your projects
- ****Documentation****: This wiki covers 90% of common scenarios

Ready to build your CodePod? Let's start with understanding what makes this little server so powerful.

****Next****: [What Is CodePod?](what-is-codepod.md)

what-is-codepod.md

layout: default

title: What Is CodePod?

sidebar: _sidebar.md

What Is CodePod?

CodePod is a DIY home server and development kit built around the Raspberry Pi Zero 2 W. It combines carefully selected hardware with a curated software stack to create a learning-focused, production-ready environment for hosting applications, experimenting with services, and understanding infrastructure.

Vision & Use Cases

For Hobbyists

- ****Home Lab****: Experiment with Docker, databases, and web services
- ****IoT Hub****: Collect and process sensor data from around your home
- ****Media Server****: Host your own streaming services with Jellyfin or Plex
- ****Network Services****: Run Pi-hole for ad-blocking, or a VPN server

For Students & Educators

- ****Learning Platform****: Hands-on experience with Linux, networking, and DevOps
- ****Portfolio Projects****: Deploy and showcase your applications
- ****Classroom Labs****: Each student gets their own server for coursework
- ****Research Projects****: Data collection and analysis on dedicated hardware

For Small Businesses

- ****Development Staging****: Test applications before production deployment

- **Internal Tools**: Host wikis, monitoring dashboards, or automation scripts
- **Backup Solutions**: Automated backups and file synchronization
- **Cost-Effective Hosting**: Run simple websites and APIs without monthly fees

Hardware & Software Summary

Hardware Foundation

- **Raspberry Pi Zero 2 W**: Quad-core ARM processor, 512MB RAM, Wi-Fi/Bluetooth
- **32GB+ microSD Card**: High-endurance card for OS and application storage
- **Power Supply**: 5V 2.5A USB-C adapter for stable operation
- **Case**: Protective enclosure with heat dissipation
- **Optional**: GPIO HATs for sensors, displays, or additional connectivity

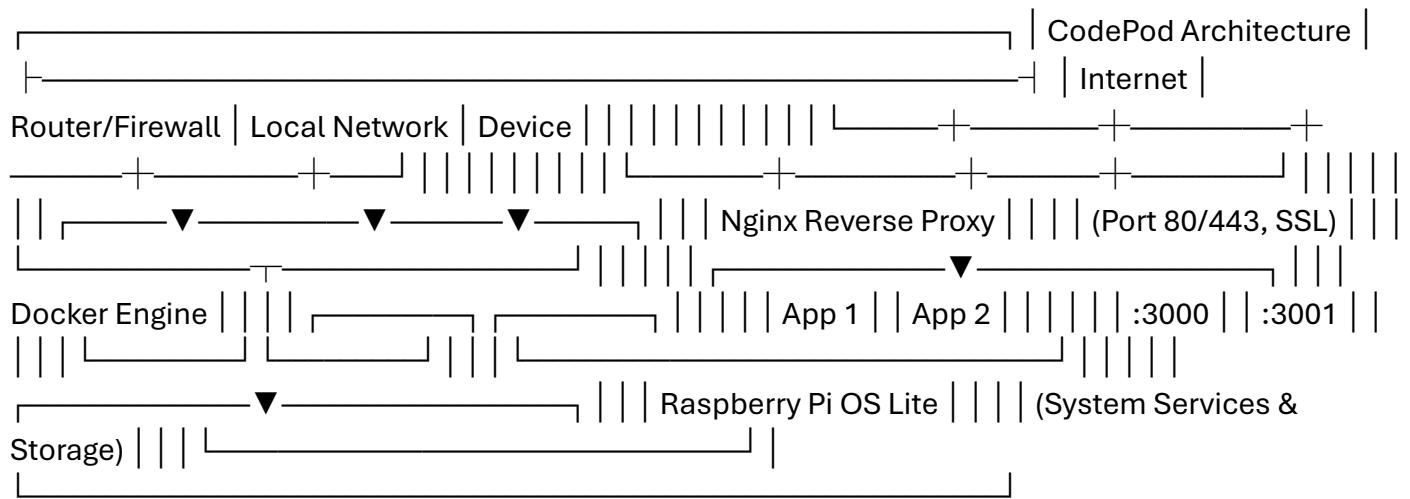
Software Stack

- **Base OS**: Raspberry Pi OS Lite (headless, minimal footprint)
- **Container Runtime**: Docker for application isolation and deployment
- **Web Server**: Nginx for reverse proxy and static content
- **Security**: UFW firewall, fail2ban, automatic security updates
- **SSL/TLS**: Let's Encrypt certificates via certbot
- **Monitoring**: Basic system monitoring and log aggregation
- **Development Tools**: Git, Node.js, Python, text editors

Key Features

- **One-Command Setup**: `codepod-setup.sh` handles initial configuration
- **Automatic Updates**: Security patches and system maintenance
- **Template System**: Pre-configured service templates for common applications
- **Resource Efficient**: Optimized for the Pi Zero 2 W's constraints
- **Documentation-First**: Every feature thoroughly documented with examples

Architecture Overview



This architecture provides:

- **External Access**: Secure HTTPS connections from anywhere
- **Service Isolation**: Each application runs in its own container
- **Resource Management**: Efficient use of limited Pi Zero 2 W resources
- **Scalability**: Easy to add new services and applications

Next: [Hardware Components](hardware-components.md)

hardware-components.md

layout: default

title: Hardware Components

sidebar: _sidebar.md

Hardware Components

Building your CodePod requires careful selection of compatible components. This guide covers everything from essential parts to optional upgrades that enhance functionality.

Bill of Materials

Essential Components

Component	Specification	Estimated Cost	Notes
-----	-----	-----	-----
Raspberry Pi Zero 2 W	1GHz quad-core, 512MB RAM	\$15	Must have Wi-Fi/Bluetooth
microSD Card	32GB+, A2/V30 rating	\$12	High-endurance recommended
Power Supply	5V 2.5A USB-C	\$8	Official Pi adapter preferred
Case	Pi Zero 2 W compatible	\$5	Heat dissipation important
microSD Card Reader	USB 3.0+	\$5	For initial setup

****Total Essential Cost: ~\$45****

Optional Upgrades

Component	Purpose	Cost	Compatibility
-----	-----	-----	-----
GPIO HAT	Sensor/display connections	\$10-30	Verify Pi Zero form factor
Heat Sink Kit	Improved cooling	\$5	Essential for heavy workloads
Mini HDMI Cable	Direct display connection	\$8	For troubleshooting
USB Hub	Multiple device connections	\$12	Micro-USB OTG required
External Storage	Additional capacity	\$20-50	USB or NAS integration

Recommended Brands

****microSD Cards:****

- SanDisk Extreme Pro (A2, V30)
- Samsung EVO Select (A2, V30)
- Kingston Canvas Go Plus (A2, V30)

⚠️ ****Avoid****: Generic or counterfeit cards. They cause corruption and performance issues.

****Power Supplies:****

- Official Raspberry Pi USB-C Power Supply
- Anker PowerPort III Nano
- UGREEN 20W USB-C Charger

💡 ****Power Tip****: Insufficient power causes random crashes and SD card corruption.

Assembly Guide

Step 1: Prepare Your Workspace

1. ****Clean Work Area****: Static-free surface with good lighting
2. ****Gather Tools****: Small screwdriver, anti-static wrist strap (optional)
3. ****Component Check****: Verify all parts are present and undamaged

Step 2: Prepare the microSD Card

```
```bash
```

```
On your computer, not the Pi
```

```
Download Raspberry Pi Imager
```

```
Select "Raspberry Pi OS Lite (32-bit)"
```

```
Configure SSH, Wi-Fi, and user account
```

```
Flash to microSD card
```

### **Step 3: Install Heat Dissipation**

1. **Clean the SoC**: Remove any residue from the processor
2. **Apply Thermal Pad**: Peel backing and center over processor
3. **Attach Heat Sink**: Press firmly and hold for 10 seconds



4. **Check Clearance:** Ensure case will fit with heat sink installed

#### Step 4: Insert microSD Card

1. **Orient Correctly:** Contacts facing down, toward the board
2. **Gentle Pressure:** Push until it clicks into place
3. **Verify Seating:** Card should be flush with the board edge

#### Step 5: Install in Case

1. **Bottom Half:** Place Pi Zero 2 W in case bottom
2. **Port Alignment:** Ensure all ports are accessible
3. **Top Half:** Attach case top, avoiding pressure on components
4. **Secure Screws:** Tighten evenly, don't overtighten

#### Step 6: Initial Power-On Test

1. **Connect Power:** Use only the recommended power supply
2. **Watch LEDs:** Green LED should flash (SD card activity)
3. **Wait for Boot:** First boot takes 2-3 minutes
4. **Check Network:** Pi should appear in your router's device list

#### Assembly Photos

*Completed CodePod showing all components properly installed*

*GPIO pins remain accessible for future expansions*

*Proper heat dissipation prevents thermal throttling*

#### Troubleshooting Assembly Issues

##### Pi Won't Boot

- **Check Power Supply:** Must be 5V 2.5A minimum
- **Verify SD Card:** Try reflashing the OS image
- **Inspect Connections:** Ensure microSD is fully inserted

##### Overheating

- **Add Heat Sink:** Essential for continuous operation

- **Improve Airflow:** Ensure case has ventilation
- **Monitor Temperature:** Use `vcgencmd measure_temp`

## GPIO Issues

- **Check Continuity:** Use multimeter to verify connections
- **Pin Mapping:** Confirm correct GPIO pin assignments
- **Power Limits:** GPIO pins have current limitations

## Next Steps

With your CodePod assembled, you're ready to configure the software stack. The next section covers the operating system and essential services that make your Pi Zero 2 W a powerful server.

---

**Next:** [Software Stack](#)

---

## software-stack.md

```markdown

layout: default

title: Software Stack

sidebar: _sidebar.md

Software Stack

CodePod's software stack balances functionality with the Pi Zero 2 W's resource constraints. Every component is chosen for efficiency, security, and educational value.

OS Recommendation

Raspberry Pi OS Lite (32-bit)

****Why This Choice?****

- ****Minimal Footprint****: No desktop environment, maximum resources for services
- ****Hardware Optimization****: Specifically tuned for Raspberry Pi hardware
- ****Long-term Support****: Regular security updates and community support
- ****32-bit Compatibility****: Better performance on Pi Zero 2 W's ARM11 architecture

****Alternative Options:****

- ****Ubuntu Server****: More software packages, higher resource usage
- ****Alpine Linux****: Extremely minimal, steeper learning curve
- ****DietPi****: Pre-optimized for Pi, less educational value

 ****64-bit Warning****: While Pi Zero 2 W supports 64-bit, the 32-bit OS performs better with limited RAM.

Preinstalled Tools

System Foundation

```
```bash
```

```
Essential system packages installed during setup
```

```
sudo apt update && sudo apt install -y \
```

```
curl \
```

```
wget \
```

```
git \
```

```
htop \
```

```
nano \
```

```
vim \
```

```
tree \
```

```
unzip \
```

```
ca-certificates \
```

```
gnupg \
```

```
lsb-release
```

## Docker Engine

**Version:** Latest stable from Docker's official repository

### Why Docker?

- **Resource Isolation:** Prevents applications from interfering
- **Easy Deployment:** Deploy complex applications with single commands
- **Version Management:** Run multiple versions of software simultaneously
- **Learning Value:** Industry-standard containerization technology

### Installation Method:

```
Automated installation via convenience script
```

```
curl -fsSL https://get.docker.com -o get-docker.sh
```

```
sudo sh get-docker.sh
```

```
sudo usermod -aG docker $USER
```

## Nginx Web Server

**Configuration:** Reverse proxy with SSL termination

### Features:

- **High Performance:** Excellent for serving static content
- **Low Memory Usage:** Efficient on resource-constrained systems
- **Flexible Configuration:** Easy to add new services
- **Security:** Built-in rate limiting and security headers

### Configuration Files:

```
/etc/nginx/sites-available/default
```

```
server{
```

```
 listen 80;
```

```
 server_name _;
```

```
 return 301 https://$server_name$request_uri;
```

```
}
```

```
server{
```

```
 listen 443 ssl http2;
```

```
server_name your-domain.com;
```

```
ssl_certificate /etc/letsencrypt/live/your-domain.com/fullchain.pem;
```

```
ssl_certificate_key /etc/letsencrypt/live/your-domain.com/privkey.pem;
```

```
location / {
```

```
 proxy_pass http://localhost:3000;
```

```
 proxy_set_header Host $host;
```

```
 proxy_set_header X-Real-IP $remote_addr;
```

```
}
```

```
}
```

## **Certbot (Let's Encrypt)**

**Purpose:** Automatic SSL certificate management

**Benefits:**

- **Free Certificates:** No cost for SSL/TLS encryption
- **Automatic Renewal:** Certificates update automatically
- **Multiple Domains:** Support for subdomains and wildcards
- **Industry Standard:** Trusted by all major browsers

## **Git Version Control**

**Configuration:** Pre-configured with common settings

```
Global Git configuration
```

```
git config --global user.name "Your Name"
```

```
git config --global user.email "your.email@example.com"
```

```
git config --global init.defaultBranch main
```

```
git config --global pull.rebase false
```

## **Security Tools**

**UFW (Uncomplicated Firewall):**

```
Default CodePod firewall rules
```

```
sudo ufw default deny incoming
```

```
sudo ufw default allow outgoing
```

```
sudo ufw allow ssh
```

```
sudo ufw allow 80/tcp
sudo ufw allow 443/tcp
sudo ufw --force enable
```

### **Fail2Ban:**

- **SSH Protection:** Blocks brute-force attacks
- **Nginx Protection:** Prevents HTTP-based attacks
- **Custom Rules:** Configurable for specific services

### **Optional Packages**

#### **Development Environments**

##### **Node.js & npm:**

```
Install Node.js via NodeSource repository
curl -fsSL https://deb.nodesource.com/setup_lts.x | sudo -E bash -
sudo apt-get install -y nodejs
```

##### **Python 3 & pip:**

```
Python development environment
sudo apt install -y python3-pip python3-venv python3-dev
python3 -m pip install --upgrade pip
```

##### **Go Programming Language:**

```
Install Go for efficient system programming
wget https://go.dev/dl/go1.21.linux-armv6l.tar.gz
sudo tar -C /usr/local -xzf go1.21.linux-armv6l.tar.gz
echo 'export PATH=$PATH:/usr/local/go/bin' >> ~/.bashrc
```

#### **Database Systems**

##### **SQLite:**

- **Lightweight:** Perfect for small to medium applications
- **Zero Configuration:** No server setup required
- **ACID Compliant:** Reliable for production use

##### **MariaDB:**

```
Install MariaDB for more complex applications
sudo apt install -y mariadb-server mariadb-client
sudo mysql_secure_installation
```

## **Redis:**

# In-memory data store for caching

```
sudo apt install -y redis-server
```

```
sudo systemctl enable redis-server
```

## **Monitoring & Logging**

### **Prometheus Node Exporter:**

# System metrics collection

```
docker run -d \
```

```
--name node-exporter \
```

```
--restart unless-stopped \
```

```
--net="host" \
```

```
--pid="host" \
```

```
-v "/:/host:ro,rslave" \
```

```
prom/node-exporter:latest \
```

```
--path.rootfs=/host
```

### **Log Management:**

# Configure log rotation

```
sudo tee /etc/logrotate.d/codepod << EOF
```

```
/var/log/codepod/*.log {
```

```
 daily
```

```
 rotate 7
```

```
 compress
```

```
 delaycompress
```

```
 missingok
```

```
 notifempty
```

```
 create 644 root root
```

```
}
```

```
EOF
```

## **Resource Optimization**

### **Memory Management**

### **Swap Configuration:**

# Optimize swap for microSD card longevity

sudo dphys-swapfile swapoff

sudo sed -i 's/CONF\_SWAPSIZE=100/CONF\_SWAPSIZE=512/' /etc/dphys-swapfile

sudo dphys-swapfile setup

sudo dphys-swapfile swapon

### **Memory Monitoring:**

# Add to .bashrc for easy memory checking

alias memcheck='free -h && echo "---" && docker stats --no-stream'

### **CPU Optimization**

#### **Governor Settings:**

# Set CPU governor for better performance

echo 'performance' | sudo tee /sys/devices/system/cpu/cpu\*/cpufreq/scaling\_governor

### **Temperature Monitoring:**

# Monitor CPU temperature

alias temp='vcgencmd measure\_temp'

### **Software Update Strategy**

#### **Automatic Security Updates**

# Install unattended-upgrades

sudo apt install -y unattended-upgrades

sudo dpkg-reconfigure -plow unattended-upgrades

#### **Docker Image Updates**

# Create update script

cat > ~/update-containers.sh << 'EOF'

#!/bin/bash

docker images --format "table {{.Repository}}\t{{.Tag}}" | grep -v REPOSITORY | while read repo tag; do

echo "Updating \$repo:\$tag"

docker pull "\$repo:\$tag"

done

docker system prune -f

EOF

chmod +x ~/update-containers.sh



Performance Benchmarks

Expected Performance

Service	Memory Usage	CPU Usage	Notes
Base OS	~80MB	1-2%	Idle system
Docker	~20MB	0-1%	Daemon only
Nginx	~5MB	0-1%	Light traffic
Node.js App	~30-50MB	5-10%	Simple application
MariaDB	~100MB	2-5%	Small database

Optimization Tips

💡 **Memory:** Use Alpine-based Docker images when possible    💡 **Storage:** Regular cleanup of Docker images and logs    💡 **Network:** Use local DNS caching for better performance    💡 **Monitoring:** Set up alerts for resource usage spikes

---

Next: [Getting Started in 5 Minutes](#)

---

## getting-started.md

```markdown

layout: default

title: Getting Started in 5 Minutes

sidebar: _sidebar.md

Getting Started in 5 Minutes

This quick-start guide gets your CodePod operational in just five minutes. Follow these steps exactly, and you'll have a working development server ready for your first application.

Prerequisites Checklist

Before starting, ensure you have:

- [] Raspberry Pi Zero 2 W with microSD card (32GB+)
- [] Reliable power supply (5V 2.5A USB-C)
- [] Computer with microSD card reader
- [] Wi-Fi network credentials
- [] 5 minutes of uninterrupted time

Step 1: Flash OS onto microSD Card

Download Raspberry Pi Imager

****On Windows/Mac/Linux:****

1. Visit [rpi.org/software](https://www.raspberrypi.org/software/)
2. Download and install Raspberry Pi Imager
3. Launch the application

Configure and Flash

1. ****Choose OS****: Select "Raspberry Pi OS Lite (32-bit)"
2. ****Choose Storage****: Select your microSD card
3. ****Configure Settings**** (click gear icon):

Enable SSH: Yes Username: codepod Password: [choose a secure password] Configure Wi-Fi: Yes
SSID: [your network name] Password: [your Wi-Fi password] Wi-Fi Country: [your country code]

4. ****Write****: Click "Write" and wait for completion (~3-5 minutes)

 ****Important****: Don't skip the configuration step! It saves significant setup time.

Step 2: First Boot & SSH Enable

Boot Your Pi

1. ****Insert microSD****: Place card in Pi Zero 2 W
2. ****Connect Power****: Use only recommended power supply
3. ****Wait for Boot****: First boot takes 2-3 minutes
4. ****Find IP Address****: Check your router's device list for "codepod"

Connect via SSH

```
```bash
```

```
Replace 192.168.1.100 with your Pi's actual IP
```

```
ssh codepod@192.168.1.100
```

```
If prompted about host authenticity, type 'yes'
```

```
Enter the password you set during imaging
```

💡 **Can't find IP?**: Try ping codepod.local or check your router's admin panel.

### Step 3: Network Configuration Verification

#### Test Internet Connectivity

```
Verify internet access
```

```
ping -c 3 google.com
```

```
Check DNS resolution
```

```
nslookup github.com
```

```
Update package lists
```

```
sudo apt update
```

#### Configure Static IP (Optional)

```
Edit dhcpcd configuration
```

```
sudo nano /etc/dhcpcd.conf
```

# Add at the end (adjust for your network):

```
interface wlan0
```

```
static ip_address=192.168.1.100/24
```

```
static routers=192.168.1.1
```

```
static domain_name_servers=8.8.8.8 1.1.1.1
```

#### **Step 4: Run codepod-setup.sh**

##### **Download and Execute Setup Script**

# Download the CodePod setup script

```
curl -fsSL https://raw.githubusercontent.com/codepod/codepod/main/setup/codepod-setup.sh -o
codepod-setup.sh
```

# Make it executable

```
chmod +x codepod-setup.sh
```

# Run the setup (takes 3-5 minutes)

```
./codepod-setup.sh
```

##### **What the Setup Script Does**

The codepod-setup.sh script automatically:

1. **Updates System:** Installs security patches and updates
2. **Installs Docker:** Adds Docker engine and configures permissions
3. **Configures Nginx:** Sets up reverse proxy with basic configuration
4. **Installs Certbot:** Prepares SSL certificate management
5. **Sets Up Firewall:** Configures UFW with secure defaults
6. **Installs Tools:** Adds essential development tools
7. **Optimizes Performance:** Tunes system for Pi Zero 2 W

##### **Setup Script Content**

```
#!/bin/bash
```

```
codepod-setup.sh - CodePod Automatic Setup Script
```

```
set -e
```

```
echo " 🚀 Starting CodePod Setup..."
```

```
Update system
```

```
echo " 📦 Updating system packages..."
```

```
sudo apt update && sudo apt upgrade -y
```

```
Install essential packages
```

```
echo " 🛠 Installing essential tools..."
```

```
sudo apt install -y curl wget git htop nano vim tree unzip \
 ca-certificates gnupg lsb-release ufw fail2ban
```

```
Install Docker
```

```
echo " 🐳 Installing Docker..."
```

```
curl -fsSL https://get.docker.com -o get-docker.sh
```

```
sudo sh get-docker.sh
```

```
sudo usermod -aG docker $USER
```

```
rm get-docker.sh
```

```
Install Docker Compose
```

```
echo " 🧑 Installing Docker Compose..."
```

```
sudo pip3 install docker-compose
```

```
Install Nginx
```

```
echo " 🌐 Installing Nginx..."
```

```
sudo apt install -y nginx
```

```
sudo systemctl enable nginx
```

```
Install Certbot
```

```
echo " 🔒 Installing Certbot..."
```

```
sudo apt install -y certbot python3-certbot-nginx
```

# Configure Firewall

echo " 🔧 Configuring firewall..."

sudo ufw default deny incoming

sudo ufw default allow outgoing

sudo ufw allow ssh

sudo ufw allow 80/tcp

sudo ufw allow 443/tcp

sudo ufw --force enable

# Configure fail2ban

echo " 🚫 Configuring fail2ban..."

sudo cp /etc/fail2ban/jail.conf /etc/fail2ban/jail.local

sudo systemctl enable fail2ban

sudo systemctl start fail2ban

# Create CodePod directory structure

echo " 📁 Creating directory structure..."

mkdir -p ~/codepod/{apps,configs,logs,scripts}

# Set up environment file

echo " ⚙️ Creating environment configuration..."

cat > ~/codepod/.env << 'EOF'

# CodePod Configuration

CODEPOD\_DOMAIN=localhost

CODEPOD\_EMAIL=admin@localhost

CODEPOD\_TIMEZONE=UTC

DOCKER\_COMPOSE\_VERSION=v2.21.0

EOF

# Create status script

echo " 🇮🇹 Creating status script..."

cat > ~/codepod/scripts/status.sh << 'EOF'

#!/bin/bash

echo "=== CodePod Status ==="

echo "Uptime: \$(uptime -p)"

echo "Memory: \$(free -h | grep Mem | awk '{print \$3 "/" \$2}')

echo "Disk: \$(df -h / | tail -1 | awk '{print \$3 "/" \$2 " (" \$5 " used)}')

echo "Temperature: \$(vcgencmd measure\_temp)"

echo "Docker: \$(docker --version)"

echo "Nginx: \$(nginx -v 2>&1)"

echo "Active Containers: \$(docker ps --format 'table {{.Names}}\t{{.Status}}' | tail -n +2)"

EOF

chmod +x ~/codepod/scripts/status.sh

# Add aliases to .bashrc

echo " 🔑 Adding helpful aliases..."

cat >> ~/.bashrc << 'EOF'

# CodePod Aliases

alias codepod-status='~/codepod/scripts/status.sh'

alias codepod-logs='sudo journalctl -u nginx -f'

alias codepod-update='sudo apt update && sudo apt upgrade -y'

alias docker-cleanup='docker system prune -f'

alias temp='vcgencmd measure\_temp'

alias memcheck='free -h'

EOF

# Optimize for Pi Zero 2 W

echo " ⚡ Optimizing for Pi Zero 2 W..."

echo 'vm.swappiness=10' | sudo tee -a /etc/sysctl.conf

```
echo 'vm.vfs_cache_pressure=50' | sudo tee -a /etc/sysctl.conf
```

```
Configure log rotation
```

```
echo " 📄 Configuring log rotation..."
```

```
sudo tee /etc/logrotate.d/codepod << 'EOF'
```

```
/home/codepod/codepod/logs/*.log {
```

```
 daily
```

```
 rotate 7
```

```
 compress
```

```
 delaycompress
```

```
 missingok
```

```
 notifempty
```

```
 create 644 codepod codepod
```

```
}
```

```
EOF
```

```
echo " ✅ CodePod setup complete!"
```

```
echo " 🔄 Please reboot your Pi to complete the installation:"
```

```
echo " sudo reboot"
```

```
echo ""
```

```
echo "After reboot, run: codepod-status"
```

## Step 5: Verify Installation

### Reboot and Test

```
Reboot to apply all changes
```

```
sudo reboot
```

```
Wait 2-3 minutes, then reconnect
```

```
ssh codepod@192.168.1.100
```

```
Run status check
```

```
codepod-status
```



## **Expected Output**

=== CodePod Status ===

Uptime: up 2 minutes

Memory: 156M/483M

Disk: 2.1G/29G (8% used)

Temperature: temp=45.7'C

Docker: Docker version 24.0.6, build ed223bc

Nginx: nginx version: nginx/1.18.0 (Ubuntu)

Active Containers:

## **Verify Services**

# Check Docker

docker run hello-world

# Check Nginx

curl -I http://localhost

# Check firewall

sudo ufw status

# Check system resources